

Calcolo Parallelo e Distribuito

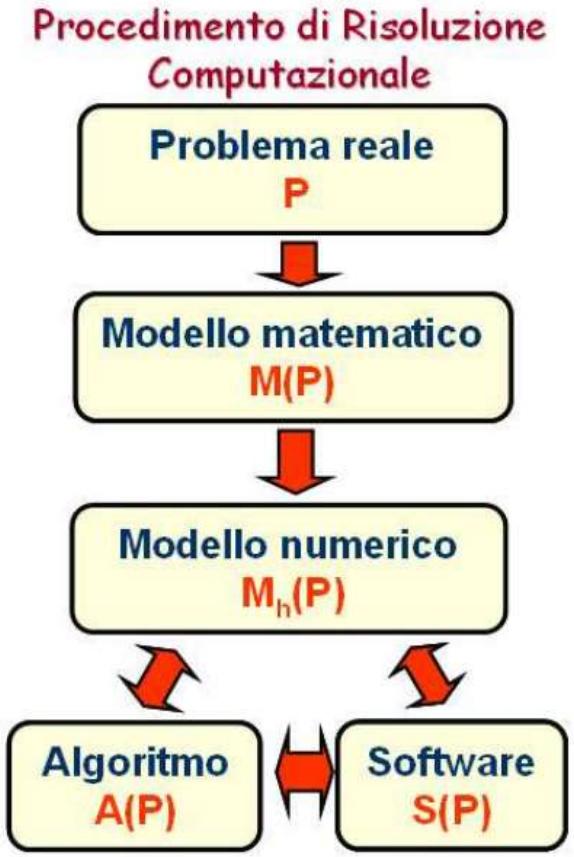
Convoluzione monodimensionale
in ambiente multicore (MIMD-SM)

Docente: Prof. L. Marcellino
Tutor: Prof. P. De Luca

Modellizzazione di problemi su larga scala



- Ricerca su internet
- Trasporto
- Pubblicità e Marketing
- Servizi bancari e finanziari
- Media e intrattenimento
- Meteorologia
- Assistenza sanitaria
- Sicurezza informatica
- Formazione



Definizione e Applicazioni della Convoluzione

- La convoluzione è un'operazione fondamentale nell'analisi delle funzioni, che combina due funzioni f e g per produrre una terza funzione h.

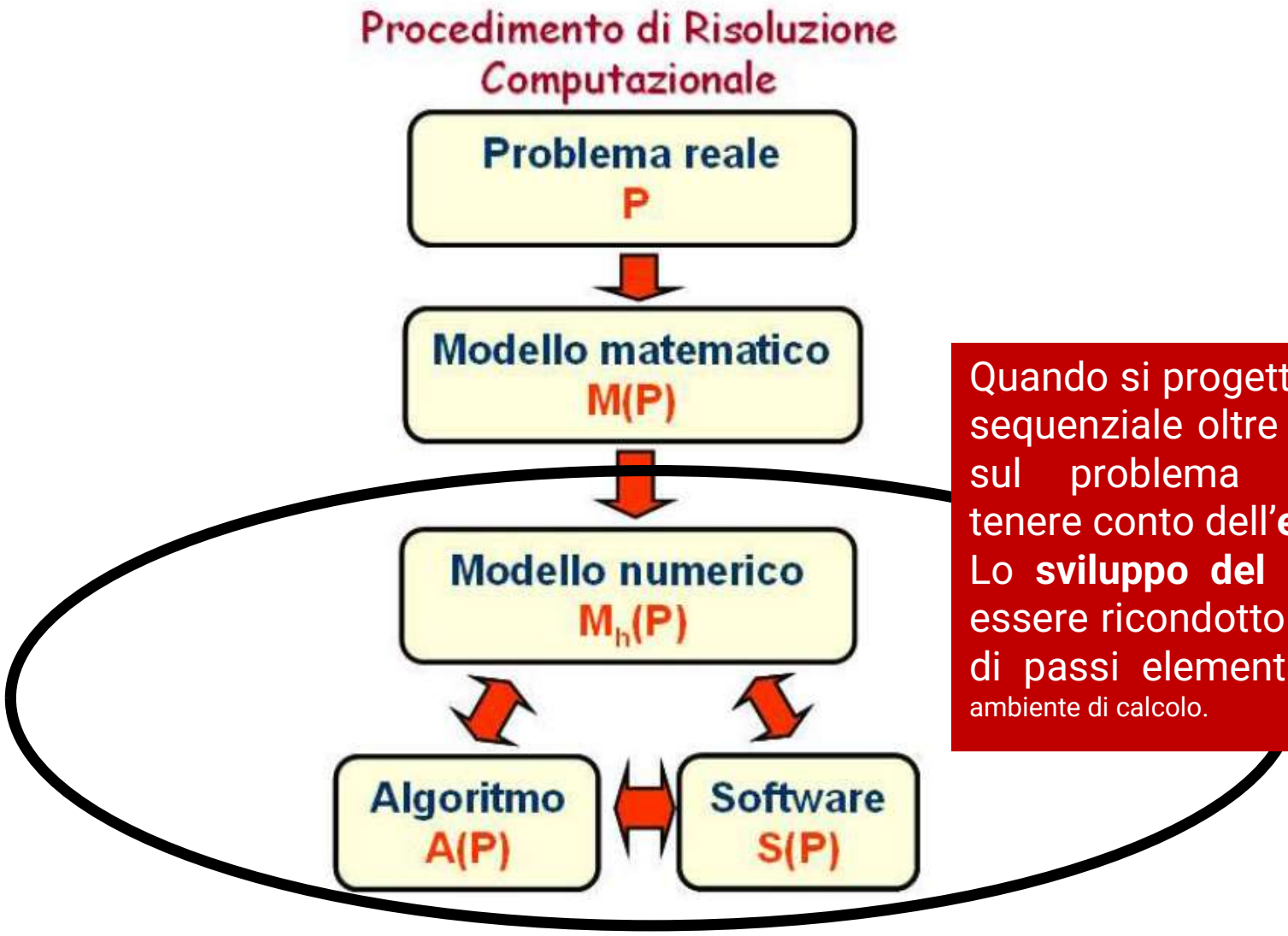
Definizione del problema

- Consideriamo x come una variabile indipendente che rappresenta il punto specifico nel quale vogliamo valutare il risultato della convoluzione delle due funzioni.
- Definiamo $f : \mathbb{R} \to \mathbb{R}, \quad \tau \mapsto f(\tau)$, che spesso rappresenta un segnale o una risposta di impulso. Nel dominio della convoluzione, consideriamo f generalmente come il segnale di input.
- Analogamente a $f(\tau)$, $g(\tau)$ è un'altra funzione di τ, spesso interpretata come la funzione di risposta di un sistema o un filtro applicato a f.

Definizione nel contesto continuo

$$(f * g)(x) = \int_{-\infty}^{\infty} f(\tau) g(x - \tau)\, d\tau$$

Modellizzazione di problemi su larga scala



Quando si progetta un algoritmo sequenziale oltre a concentrarsi sul problema è importante tenere conto dell'**esecutore**.
Lo **sviluppo del software** deve essere ricondotto ad un insieme di passi elementari, per lo specifico ambiente di calcolo.

Convoluzione con Supporto Finito

Quando le sequenze hanno un supporto finito:

$$(f * g)[n] = \sum_{m=0}^{\min(M-1,n)} f[m] \cdot g[n-m], \quad n \in [0, M+N-2]$$

Esempio Numerico

Dati $f = [3, 5, 2, 1]$ e $g = [1, 2]$, calcoliamo $(f * g)[2]$:

$$(f * g)[2] = \sum_{m=0}^{2} f[m] \cdot g[2-m] = f[0] \cdot g[2] + f[1] \cdot g[1] + f[2] \cdot g[0] = 0 + 5 \cdot 2 + 2 \cdot 1 = 12$$

Convoluzione discreta monodimensionale





Input:

- array A di lunghezza N
- array C, di raggio k, lunghezza 2k+1

Output:

- array B di lunghezza N



Convoluzione discreta monodimensionale

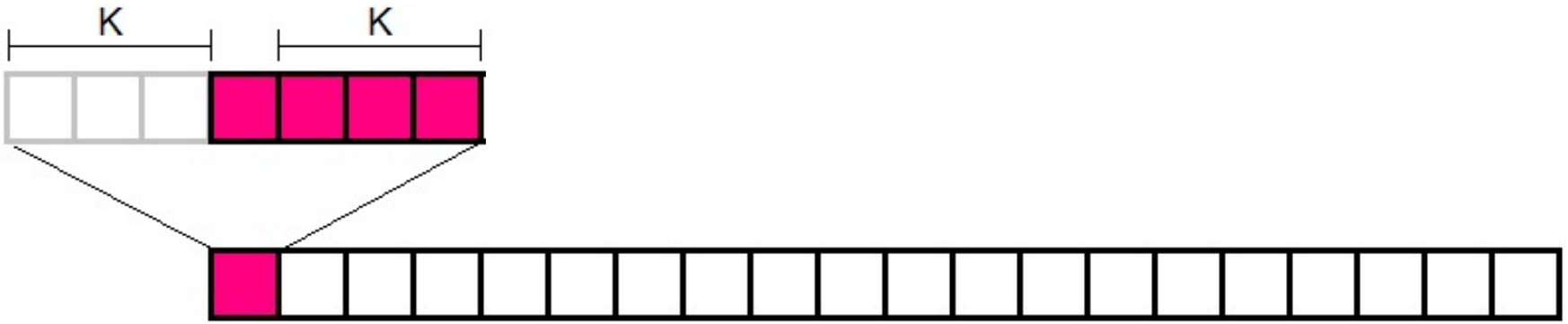


$$B[i] = \sum_{j=-k}^{k} A[i+j] \times C[j+k], i = 0, \ldots, N$$



Convoluzione monodimensionale – filtri raggio d'azione

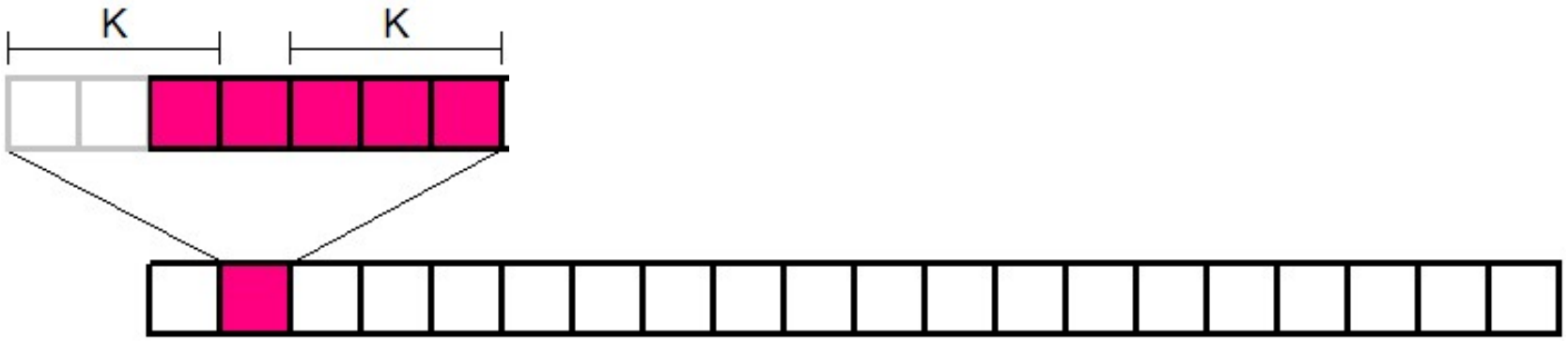
N = 20 - k = 3



$$B[i] = \sum_{j=k}^{2k+1} A[i+j] \times C[j+k],$$

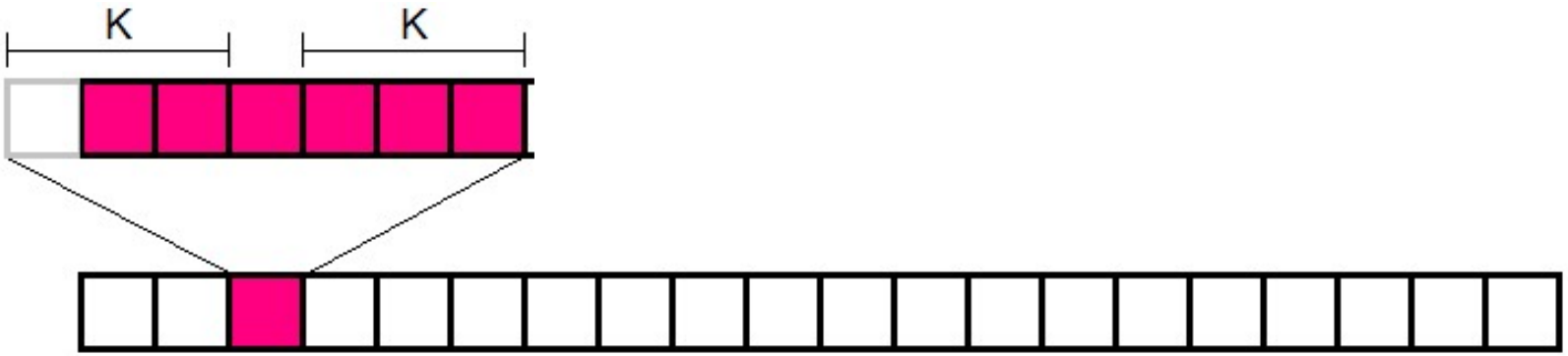
Convoluzione monodimensionale – filtri raggio d'azione

N = 20 - k = 3



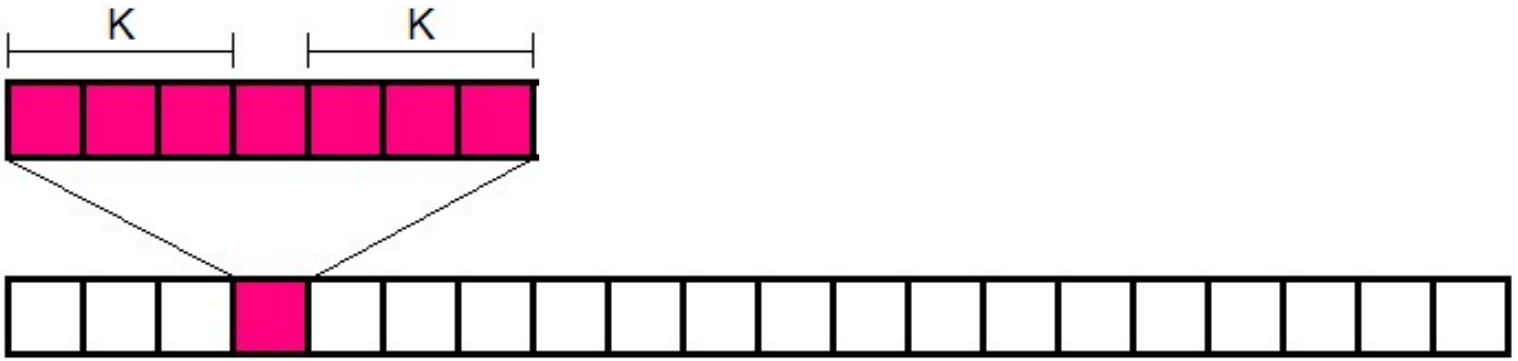
Convoluzione monodimensionale – filtri raggio d'azione

N = 20 - k = 3



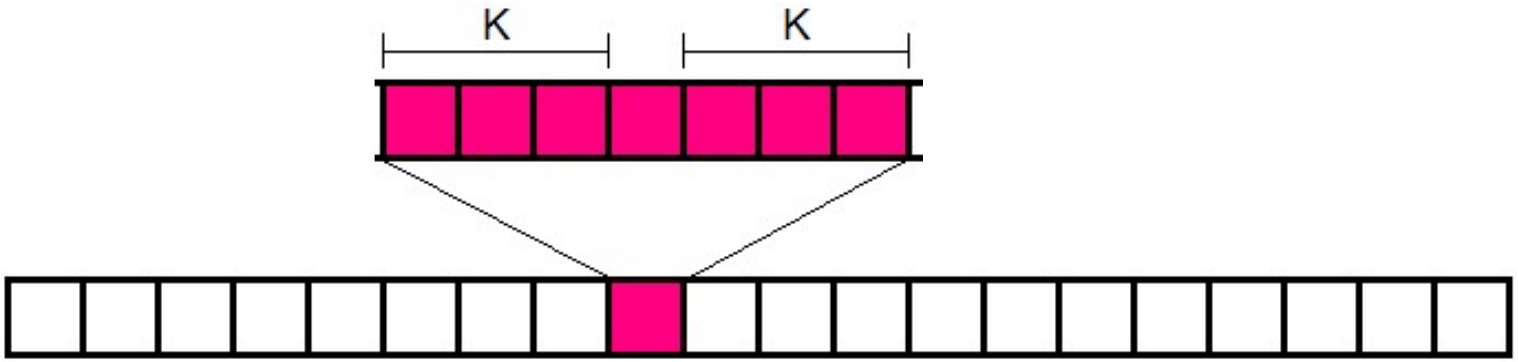
Convoluzione monodimensionale – filtri raggio d'azione

N = 20 - k = 3



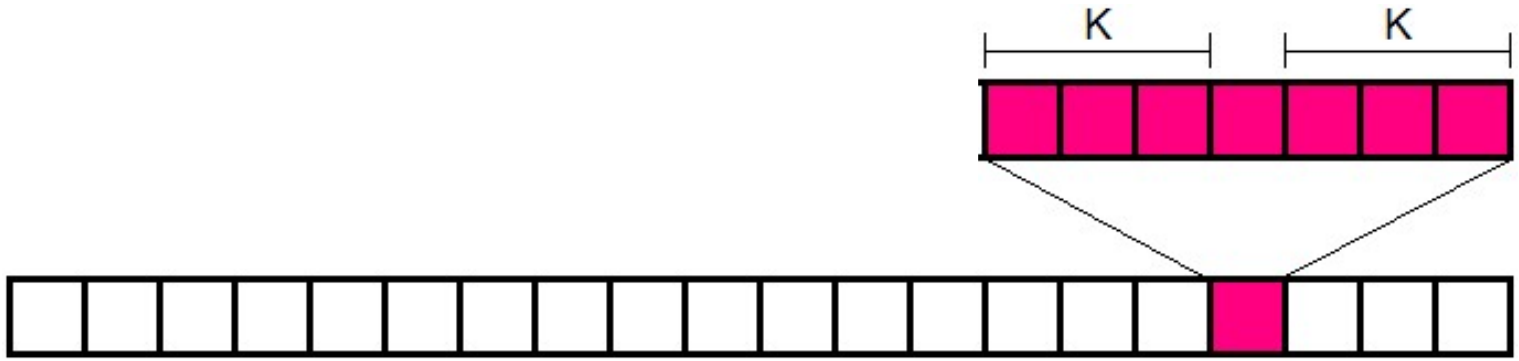
Convoluzione monodimensionale – filtri raggio d'azione

N = 20 - k = 3



Convoluzione monodimensionale – filtri raggio d'azione

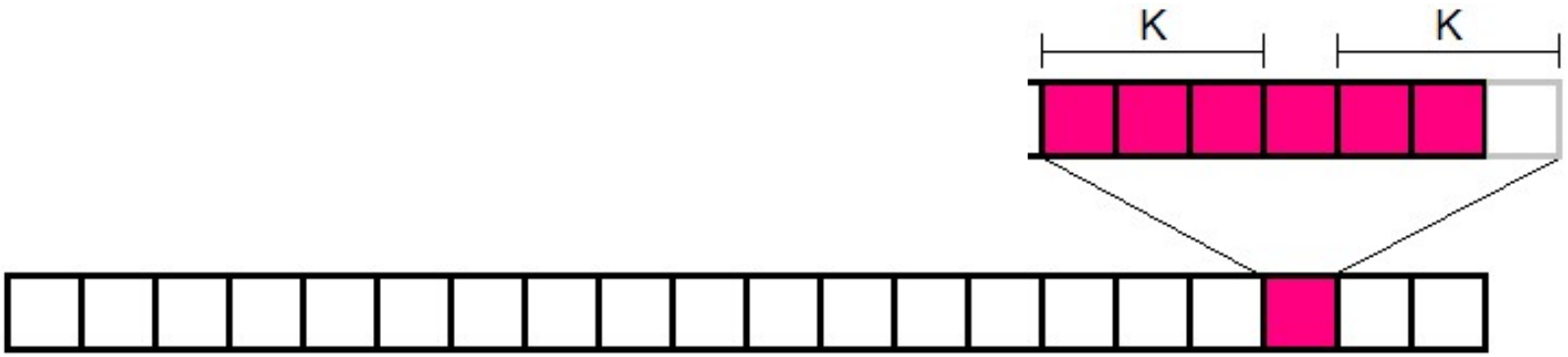
N = 20 - k = 3



Convoluzione monodimensionale – filtri

raggio d'azione

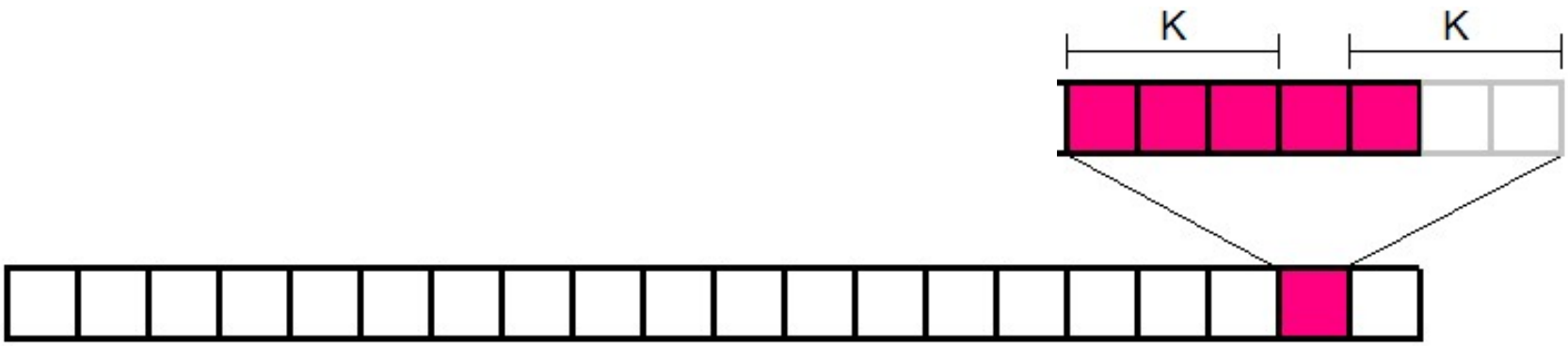
N = 20 - k = 3



Convoluzione monodimensionale – filtri

raggio d'azione

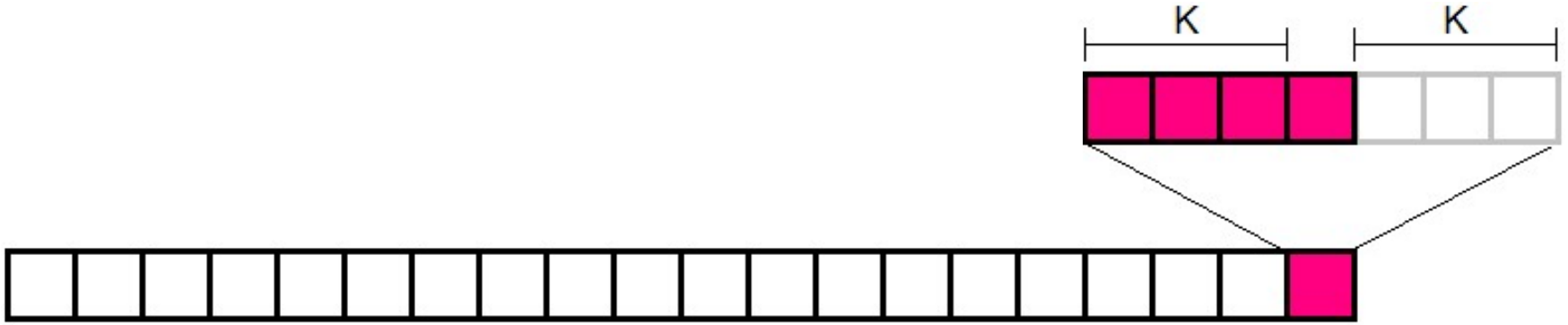
N = 20 - k = 3



Convoluzione monodimensionale – filtri

raggio d'azione

N = 20 - k = 3

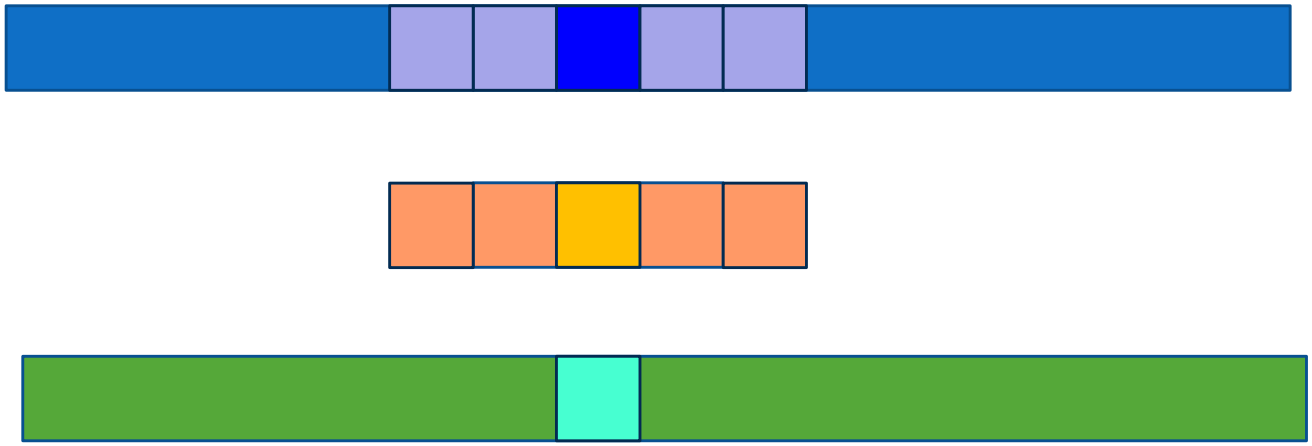


Soluzione elementare:

$$B[i] = \sum_{j=-k}^{k} A[i+k] \times C[j+k], i = k, \ldots, N-k$$

```
begin
     for  i=0;i<N;i++ do
          for (j=-k;j<=k;j++) do
             if (i+j>=0 && i+j<N ) do
              B[i]=B[i]+A[i+j]*C[j+k]
             endif
          endfor
     endofor
end
```

Esempio N=20, k=2, al passo i=10



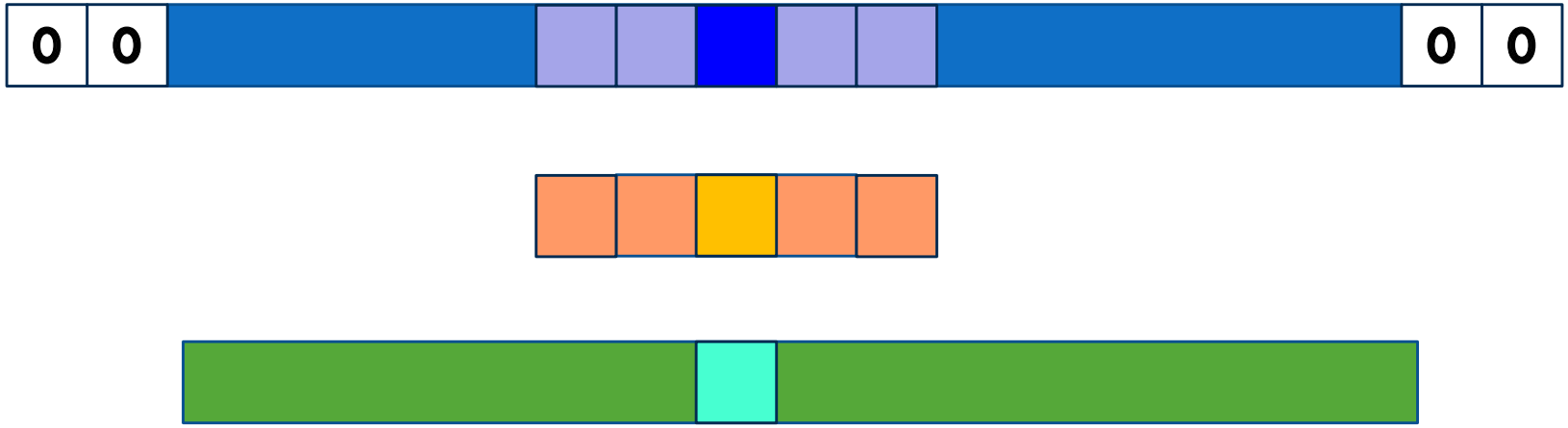
$$B[10] = \sum_{j=-2}^{2} A[i+j] \times C[j+k]$$

= A[8]*C[0] + A[9]*C[1] + A[10]*C[2] + A[11]*C[3]+ A[12]*C[4]

È un prodotto scalare!!!

Esempio N=20, k=2, al passo i=10

… se faccio: il padding



N prodotti scalari di lunghezza L=5=2k+1

Complessità computazionale sequenziale:
N dimensione del vettore iniziale,
L=2k+1 dimensione del kernel
k raggio del kernel.

dim[A]=N, dim[C]=2k+1=L

convoluzione monodimensionale

L=2k+1

Complessità computazionale sequenziale

N volte: 1 prodotto scalare tra vettori di lunghezza L

N[2L-1] operazioni

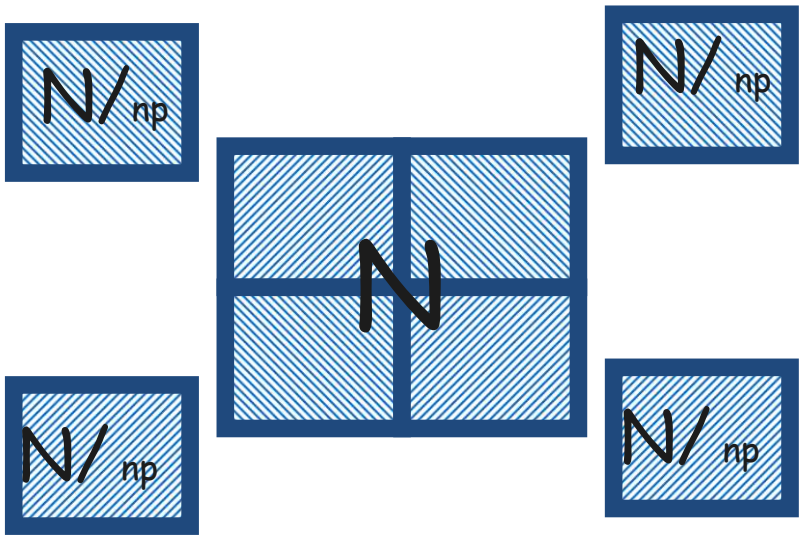
$$T_1(N,L) = N[2L-1]$$

Il parallelismo delle architetture MIMD

convoluzione monodimensionale

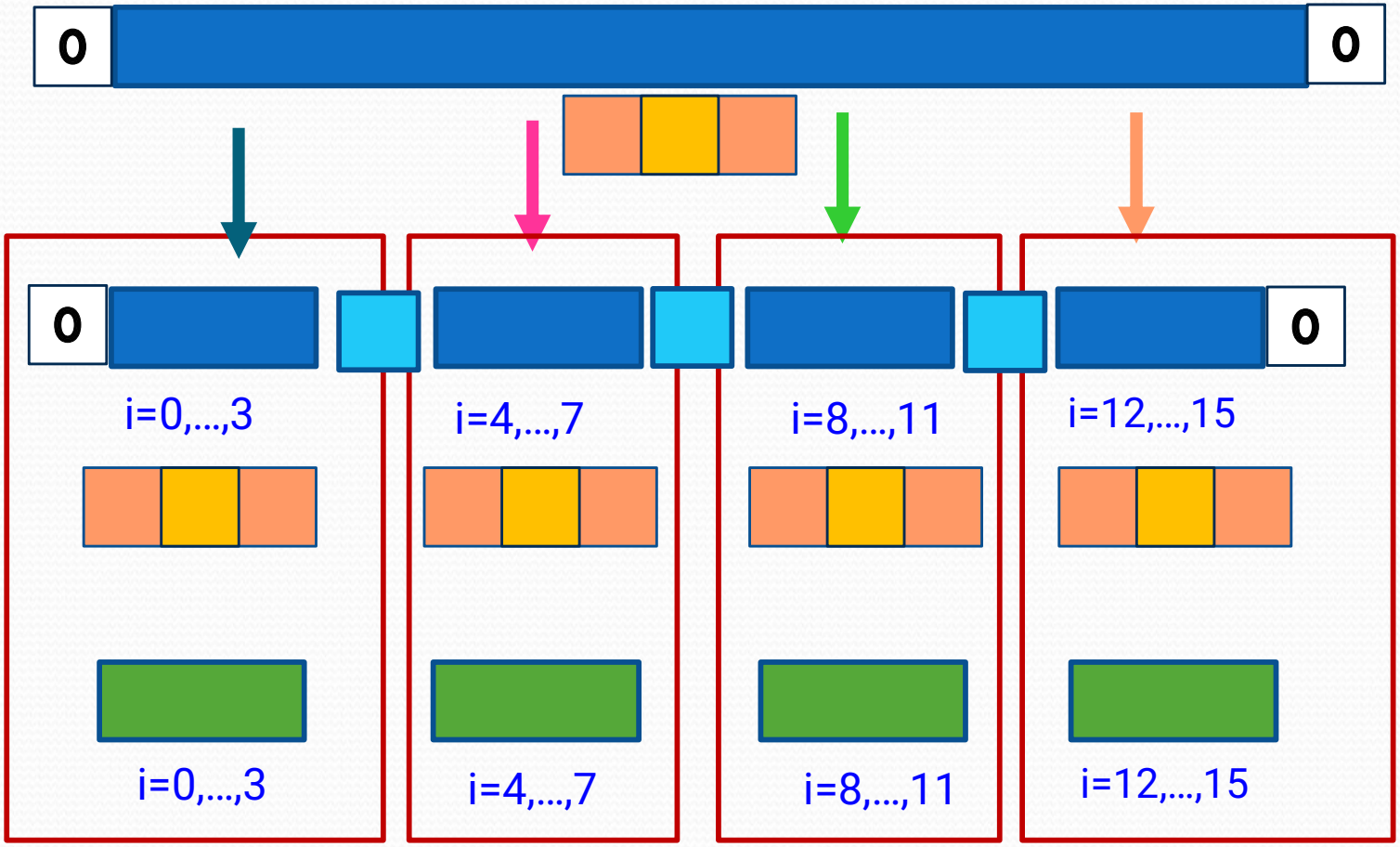
Se ho a disposizione np unità processanti, come posso procedere sfruttando il concetto di calcolo parallelo?

Decomporre un problema di dimensione N in **np** sottoproblemi di dimensione N/**np** e risolverli contemporaneamente usando **np** CPU



Strategia di parallelizzazione
convoluzione monodimensionale

Esempio: N=16, L=3 (k=1), np=4



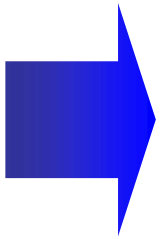
p, dim[A]=N, dim[C]=2k+1=L

convoluzione monodimensionale

Calcolo convoluzioni locali

N/p

prodotti scalari



N[2L-1]/p

…e ho finito!

$$T_p(N,L) = N[2L-1]/p$$

p, dim[A]=N, dim[C]=2k+1=L

convoluzione monodimensionale

In sequenziale

$$T_1(N,L) = N[2L-1]$$

$$S_p = T_1(N,L)/T_p(N,L) =$$

$$= N[2L-1] / (N[2L-1]/p) = p$$

$$Oh = p\,T_p(N) - T_1(N) = 0$$

$$E_p = S_p / p = 1$$

p=8, N=67, L

Cosa succede se N non è esattamente divisibile per p???

9	9	9	8	8	8	8	8

Come se tutti i processori
avessero un vettore di
lunghezza 9 da elaborare

$T_p(N,L)$= ???

isoefficienza

Definizione

L'ISOEFFICIENZA

è una funzione di tre variabili p_0 , p_1 , n_0

e definisce la costante che lega la nuova dimensione del problema da scegliere n_1 per valutare la **scalabilità** di un algortimo

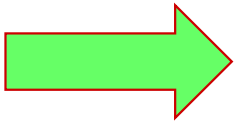
$$I(p_0 , p_1 , n_0) = O_h(p_1 , n_1)/O_h(p_0 , n_0)$$

Calcolo isoefficienza

Convoluzione monodimensionale vettore dim N – kernel dim L (k, raggio)

$$O_h(p, N, L) = T_1(N,L) - p\ T_p(N,L) =$$

$$= N[2L-1] - p[N[2L-1]/p] = 0$$

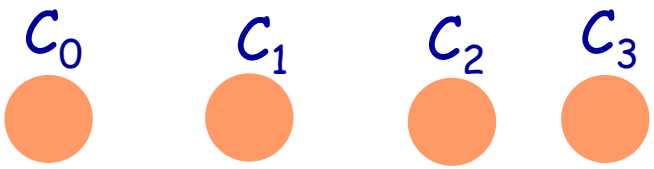


Certo!
Ho pensato ad un algoritmo full-parallel.
Sarà sicuramente scalabile

Caratterizziamo ora lo **speedup** **usando la legge di Ware-Amdahl**

base

p=4, dim[a]=N=32, nloc=8, L=5, k=2



In sequenziale N[2L-1]=288 operazioni

unica fase (tutta parallela)

nloc[2L-1] = 72 operazioni

72 operazioni fatte contemporaneamente da 4 core

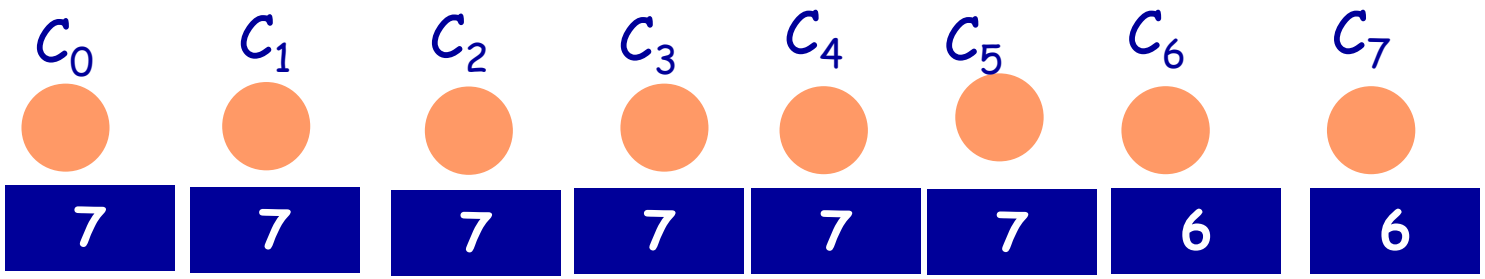
72x4=288 delle 288 op. totali

$1-\alpha = 288/288$

$\alpha = 0$

... devo continuare...?

Cosa succede se N non è esattamente divisibile per p???



p=8, N=54

Quale che sia la strategia per la collezione delle somme parziali, quello che cambia è solo α_8

Implementare il facilissimo
algoritmo parallelo per
la convoluzione monodimensionale
di un vettore di dimensione N con
kernel di raggio k